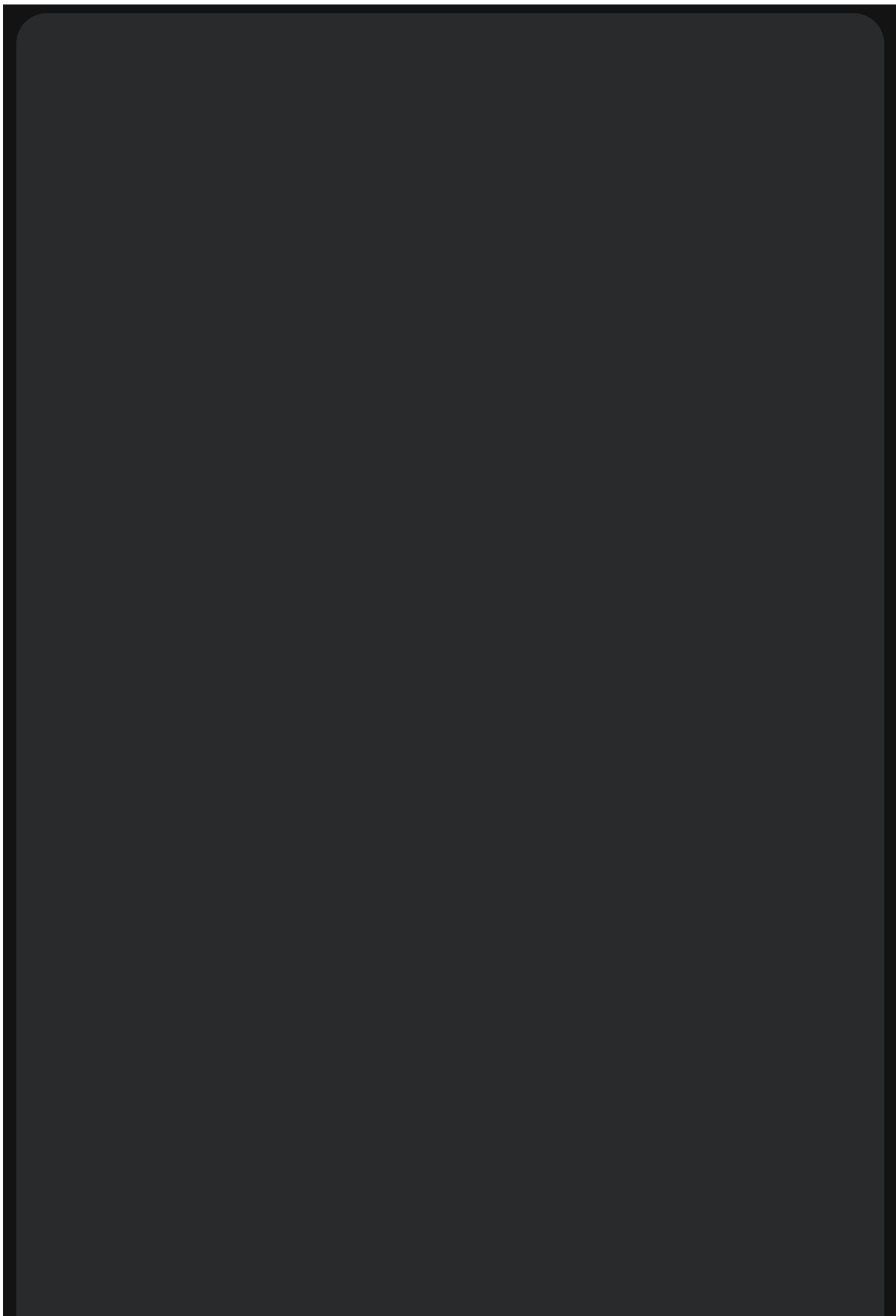




CLASSES TO BE TRAINED:

```
1 # Selected classes: fist, four, like, mute, ok, one, palm, peace, stop, three, three2
2 # None Classes: rock, call, dislike,
```

IMPORTANT: SET THE RUNTIME VERSION TO 2025.07

Install dependencies

```
1 !pip install --upgrade pip
2 !pip install mediapipe-model-maker
```

Show hidden output

```
1 import os
2 import tensorflow as tf
3 assert tf.__version__.startswith('2')
4
5 from mediapipe_model_maker import gesture_recognizer
6 import matplotlib.pyplot as plt
```

Show hidden output

```
1 # !pip install kaggle
2 # from google.colab import files
3 # files.upload() # upload your kaggle.json API token here
4 # !mkdir -p ~/.kaggle
5 # !cp kaggle.json ~/.kaggle/
6 # !chmod 600 ~/.kaggle/kaggle.json
7
8 # !kaggle datasets download -d innominate817/hagrid-sample-30k-384p
9 !unzip -q hagrid-sample-30k-384p.zip -d hagrid_30k
10
```

Show hidden output

```
1 import shutil
2
3 SUBSET = ["fist", "four", "like", "ok", "one", "palm", "peace", "stop", "three", "thr
4 # If you want swipe support later, make sure the required pairs from
5 # $2.1 are included here, e.g. add "stop_inverted" and/or "palm", "fist".
6
7 dataset_path = "gesture_dataset"
8 os.makedirs(dataset_path, exist_ok=True)
9
10 # HaGRID-30K unzips into per-class folders already – locate them and
11 # copy only the classes you chose into the training folder structure.
12 hagrid_root = "hagrid_30k/hagrid-sample-30k-384p/hagrid_30k" # adjust if the unzip p
13
14 for cls in SUBSET:
15     corrected_name = "train_val_" + cls
16     src = os.path.join(hagrid_root, corrected_name)
17     dst = os.path.join(dataset_path, corrected_name)
18     if os.path.isdir(src):
19         shutil.copytree(src, dst)
```

```

19     shutil.copytree(src, dst)
20     else:
21         print(f"WARNING: {src} not found – check the unzip folder structure")
22
23 # Build the required 'none' class from a handful of other unused HaGRID
24 # classes (or any hand-free / ambiguous images) so the model learns
25 # what "not one of my gestures" looks like. Pick source classes that
26 # are NOT in your SUBSET above.
27 none_dir = os.path.join(dataset_path, "none")
28 os.makedirs(none_dir, exist_ok=True)
29
30 NONE_SOURCE_CLASSES = ["rock", "call", "dislike", "mute"] # must not overlap with SU
31 TARGET_NONE_TOTAL = 1800
32 NONE_SAMPLES_PER_CLASS = TARGET_NONE_TOTAL // len(NONE_SOURCE_CLASSES) # = 600
33
34 print(f"Pulling {NONE_SAMPLES_PER_CLASS} images per none-source class "
35       f"({len(NONE_SOURCE_CLASSES)} classes, target total {TARGET_NONE_TOTAL})")
36
37 for cls in NONE_SOURCE_CLASSES:
38     corrected_name = "train_val_" + cls
39     src = os.path.join(hagrid_root, corrected_name)
40     if not os.path.isdir(src):
41         continue
42     for i, fname in enumerate(os.listdir(src)[:NONE_SAMPLES_PER_CLASS]):
43         shutil.copy(os.path.join(src, fname),
44                     os.path.join(none_dir, f"{corrected_name}_{fname}"))
45
46 print("Final dataset folders:", os.listdir(dataset_path))
47

```

Pulling 450 images per none-source class (4 classes, target total 1800)
 Final dataset folders: ['train_val_ok', 'train_val_four', 'train_val_palm', 'train_val_two

```

1 labels = [d for d in os.listdir(dataset_path)
2            if os.path.isdir(os.path.join(dataset_path, d))]
3 print("Labels:", labels)
4 assert "none" in labels, "Missing required 'none' class"
5
6 for label in labels:
7     n = len(os.listdir(os.path.join(dataset_path, label)))
8     print(f" {label}: {n} images")
9

```

Labels: ['train_val_ok', 'train_val_four', 'train_val_palm', 'train_val_two_up_inverted',
 train_val_ok: 1750 images
 train_val_four: 1805 images
 train_val_palm: 1770 images
 train_val_two_up_inverted: 1765 images
 train_val_fist: 1735 images
 train_val_stop: 1748 images
 train_val_three: 1751 images
 train_val_three2: 1737 images
 train_val_like: 1732 images
 train_val_one: 1778 images
 train_val_peace: 1769 images
 train_val_two_up: 1855 images
 train_val_peace_inverted: 1742 images
 train_val_stop_inverted: 1803 images
 none: 1800 images

```

1 data = gesture_recognizer.Dataset.from_folder(
2     dirname=dataset_path,
3     hparams=gesture_recognizer.HandDataPreprocessingParams()
4 )
5 train data, rest data = data.split(0.8)

```

```
6 validation_data, test_data = rest_data.split(0.5)
7
```

Downloading https://storage.googleapis.com/mediapipe-assets/palm_detection_full.tflite to /
Downloading https://storage.googleapis.com/mediapipe-assets/hand_landmark_full.tflite to /
Downloading https://storage.googleapis.com/mediapipe-assets/gesture_embedder.tar.gz to /tmp

```
1 hparams = gesture_recognizer.HParams(
2     export_dir="exported_model",
3     epochs=15,          # HaGRID subset is clean; 15 is a reasonable start
4     batch_size=8,
5     learning_rate=0.001,
6     shuffle=True,
7 )
8 options = gesture_recognizer.GestureRecognizerOptions(hparams=hparams)
9
10 model = gesture_recognizer.GestureRecognizer.create(
11     train_data=train_data,
12     validation_data=validation_data,
13     options=options
14 )
15
```

Model: "model"

Layer (type)	Output Shape	Param #
=====		
hand_embedding (InputLayer)	[(None, 128)]	0
batch_normalization (Batch Normalization)	(None, 128)	512
re_lu (ReLU)	(None, 128)	0
dropout (Dropout)	(None, 128)	0
custom_gesture_recognizer_ out (Dense)	(None, 15)	1935

=====
Total params: 2447 (9.56 KB)
Trainable params: 2191 (8.56 KB)
Non-trainable params: 256 (1.00 KB)

None

Epoch 1/15

2294/2294 [=====] - 20s 8ms/step - loss: 0.6056 - categorical_accu

Epoch 2/15

2294/2294 [=====] - 21s 9ms/step - loss: 0.4368 - categorical_accu

Epoch 3/15

2294/2294 [=====] - 20s 9ms/step - loss: 0.4070 - categorical_accu

Epoch 4/15

2294/2294 [=====] - 21s 9ms/step - loss: 0.3998 - categorical_accu

Epoch 5/15

2294/2294 [=====] - 19s 8ms/step - loss: 0.3933 - categorical_accu

Epoch 6/15

2294/2294 [=====] - 21s 9ms/step - loss: 0.3865 - categorical_accu

Epoch 7/15

2294/2294 [=====] - 21s 9ms/step - loss: 0.3807 - categorical_accu

Epoch 8/15

2294/2294 [=====] - 19s 8ms/step - loss: 0.3789 - categorical_accu

Epoch 9/15

2294/2294 [=====] - 21s 9ms/step - loss: 0.3764 - categorical_accu

Epoch 10/15

2294/2294 [=====] - 21s 9ms/step - loss: 0.3760 - categorical_accu

Epoch 11/15

```

2294/2294 [=====] - 20s 9ms/step - loss: 0.3747 - categorical_acc
Epoch 12/15
2294/2294 [=====] - 20s 9ms/step - loss: 0.3731 - categorical_acc
Epoch 13/15
2294/2294 [=====] - 21s 9ms/step - loss: 0.3737 - categorical_acc
Epoch 14/15
2294/2294 [=====] - 20s 9ms/step - loss: 0.3719 - categorical_acc
Epoch 15/15
2294/2294 [=====] - 21s 9ms/step - loss: 0.3690 - categorical_acc

```

```

1 loss, acc = model.evaluate(test_data, batch_size=1)
2 print(f"Test loss: {loss:.4f}    Test accuracy: {acc:.4f}")
3

```

```

2295/2295 [=====] - 12s 2ms/step - loss: 0.3024 - categorical_acc
Test loss: 0.3024    Test accuracy: 0.8710

```

```

1 model.export_model()
2 !ls exported_model
3

```

```

Downloading https://storage.googleapis.com/mediapipe-assets/gesture_embedder.tflite to /tmp
Using existing files at /tmp/model_maker/gesture_recognizer/palm_detection_full.tflite
Using existing files at /tmp/model_maker/gesture_recognizer/hand_landmark_full.tflite
Downloading https://storage.googleapis.com/mediapipe-assets/canned_gesture_classifier.tflite
best_model_weights.data-00000-of-00001  epoch_models      metadata.json
best_model_weights.index                gesture_recognizer.task
checkpoint                               logs

```

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.metrics import confusion_matrix, classification_report
4
5 # Get true labels and predictions on the test set
6 label_names = test_data.label_names # class names in the order Model Maker used
7 print("Classes:", label_names)
8
9 # Model Maker's Dataset object wraps a tf.data.Dataset - pull features/labels out
10 y_true = []
11 y_pred = []
12
13 for features, labels in test_data.gen_tf_dataset(batch_size=1, is_training=False):
14     pred = model._model.predict(features, verbose=0)
15     y_true.append(np.argmax(labels.numpy(), axis=1)[0])
16     y_pred.append(np.argmax(pred, axis=1)[0])
17
18 print([a for a in dir(model) if not a.startswith('__')])
19
20 y_true = np.array(y_true)
21 y_pred = np.array(y_pred)
22
23 # Print per-class precision/recall/f1
24 print("\nPer-class report:")
25 print(classification_report(y_true, y_pred, target_names=label_names, digits=3))
26
27 # Plot confusion matrix
28 cm = confusion_matrix(y_true, y_pred)
29 fig, ax = plt.subplots(figsize=(8, 7))
30 im = ax.imshow(cm, cmap="Blues")
31 ax.set_xticks(range(len(label_names)))
32 ax.set_yticks(range(len(label_names)))
33 ax.set_xticklabels(label_names, rotation=45, ha="right")
34 ax.set_yticklabels(label_names)
35 ax.set_xlabel("Predicted")

```

```

35 ax.imshow(cm, cmap=cm.cm)
36 ax.set_ylabel("True")
37 ax.set_title("Confusion Matrix")
38
39 for i in range(len(label_names)):
40     for j in range(len(label_names)):
41         ax.text(j, i, cm[i, j], ha="center", va="center",
42               color="white" if cm[i, j] > cm.max() / 2 else "black")
43
44 plt.colorbar(im)
45 plt.tight_layout()
46 plt.show()

```

Classes: ['none', 'train_val_fist', 'train_val_four', 'train_val_like', 'train_val_ok', 'train_val_one', 'train_val_palm', 'train_val_peace', 'train_val_peace_inverted', 'train_val_stop', 'train_val_stop_inverted', 'train_val_three', 'train_val_three2', 'train_val_two_up', 'train_val_two_up_inverted']

Per-class report:

	precision	recall	f1-score	support
none	0.878	0.808	0.841	151
train_val_fist	0.888	0.906	0.897	149
train_val_four	0.941	0.934	0.938	137
train_val_like	0.776	0.905	0.835	126
train_val_ok	0.973	0.894	0.932	160
train_val_one	0.936	0.842	0.886	139
train_val_palm	0.963	0.846	0.901	182
train_val_peace	0.948	0.786	0.859	140
train_val_peace_inverted	0.857	0.828	0.842	145
train_val_stop	0.817	0.907	0.860	162
train_val_stop_inverted	0.936	0.860	0.896	186
train_val_three	0.545	0.887	0.676	142
train_val_three2	0.993	0.931	0.961	160
train_val_two_up	0.948	0.872	0.908	187
train_val_two_up_inverted	0.860	0.860	0.860	129
accuracy			0.871	2295
macro avg	0.884	0.871	0.873	2295
weighted avg	0.889	0.871	0.876	2295

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.metrics import confusion_matrix, classification_report
4
5 label_names = test_data.label_names
6 print("Classes:", label_names)
7
8 y_true = []
9 y_pred = []
10
11 for features, labels in test_data.gen_tf_dataset(batch_size=1, is_training=False):
12     pred = model._model.predict(features, verbose=0)
13     y_true.append(np.argmax(labels.numpy(), axis=1)[0])
14     y_pred.append(np.argmax(pred, axis=1)[0])
15
16 y_true = np.array(y_true)
17 y_pred = np.array(y_pred)
18
19 print("\nPer-class report:")
20 print(classification_report(y_true, y_pred, target_names=label_names, digits=3))
21
22 cm = confusion_matrix(y_true, y_pred)
23 fig, ax = plt.subplots(figsize=(10, 9))
24 im = ax.imshow(cm, cmap="Blues")
25 ax.set_xticks(range(len(label_names)))
26 ax.set_yticks(range(len(label_names)))

```

```

27 ax.set_xticklabels(label_names, rotation=45, ha="right")
28 ax.set_yticklabels(label_names)
29 ax.set_xlabel("Predicted")
30 ax.set_ylabel("True")
31 ax.set_title("Confusion Matrix")
32
33 for i in range(len(label_names)):
34     for j in range(len(label_names)):
35         ax.text(j, i, cm[i, j], ha="center", va="center",
36                color="white" if cm[i, j] > cm.max() / 2 else "black")
37
38 plt.colorbar(im)
39 plt.tight_layout()
40 plt.show()

```

Classes: ['none', 'train_val_fist', 'train_val_four', 'train_val_like', 'train_val_ok', 'train_val_one', 'train_val_palm', 'train_val_peace', 'train_val_peace_inverted', 'train_val_stop', 'train_val_stop_inverted', 'train_val_three', 'train_val_three2', 'train_val_two_up', 'train_val_two_up_inverted']

Per-class report:

	precision	recall	f1-score	support
none	0.878	0.808	0.841	151
train_val_fist	0.888	0.906	0.897	149
train_val_four	0.941	0.934	0.938	137
train_val_like	0.776	0.905	0.835	126
train_val_ok	0.973	0.894	0.932	160
train_val_one	0.936	0.842	0.886	139
train_val_palm	0.963	0.846	0.901	182
train_val_peace	0.948	0.786	0.859	140
train_val_peace_inverted	0.857	0.828	0.842	145
train_val_stop	0.817	0.907	0.860	162
train_val_stop_inverted	0.936	0.860	0.896	186
train_val_three	0.545	0.887	0.676	142
train_val_three2	0.993	0.931	0.961	160
train_val_two_up	0.948	0.872	0.908	187
train_val_two_up_inverted	0.860	0.860	0.860	129
accuracy			0.871	2295
macro avg	0.884	0.871	0.873	2295
weighted avg	0.889	0.871	0.876	2295

```

1 import numpy as np
2
3 label_names = test_data.label_names
4 three_idx = label_names.index("none")
5
6 print(f"{'True class':<28} {'Predicted as three':>5}")
7 for i, name in enumerate(label_names):
8     count = cm[i, three_idx]
9     if count > 0:
10         print(f"{'name':<28} {'count':>5}")

```

True class	Predicted as three
none	122
train_val_fist	3
train_val_like	2
train_val_ok	1
train_val_one	3
train_val_peace	1
train_val_stop	1
train_val_stop_inverted	1
train_val_three	1
train_val_three2	1
train_val_two_up	2
train_val_two_up_inverted	1

